PhishLang-Web: Auditing and Deploying Fully In-Browser Phishing Detection with MobileBERT

Masroor Ul Hasan Syed Deepthi Kondreddy

CS 8027: Advanced Networking & Architecture — Research Project Proposal, Fall 2026

1 Introduction

Phishing websites imitate trusted brands to steal credentials and payment data. The dominant defence, server-side blocklists such as Google Safe Browsing, protects users only after a page has been reported, crawled and confirmed. Measurements show that blocklisting can take hours and often misses evasive pages [1, 2]. Detecting phishing *on the client* avoids the blocklist delay, keeps browsing history on the device, and keeps working when the cloud is unreachable [2]. Yet a longitudinal study found that the client-side anti-phishing measures deployed today detected only 14% of phishing websites [3]. Recent accurate detectors rely on large, typically API-hosted language models [4, 5, 6], which reintroduces the cloud, its cost and its privacy exposure.

This is a networking and systems problem as much as a machine-learning one. *Where* inference runs (the cloud, a local native service, or the browser itself) determines detection latency, what data leaves the device, installation size, and failure modes. We study the most constrained point in this design space: a detector that runs entirely inside the browser. We ask whether it can be both deployable and accurate against current phishing.

2 Motivation and Background

PhishLang. PhishLang [2] (NDSS 2026) parses a page's HTML into a compact text representation of 18 security-relevant tag types (forms, inputs, links, scripts, iframes, ...). It then fine-tunes MobileBERT [7], a 25M-parameter BERT variant built for resource-limited devices, on the Phishpedia dataset [8]. The authors report 0.96 accuracy and F1, a 0.39 s median inference time and 74 MB of memory. They also harden the model against problem-space adversarial HTML attacks [9]. Two open issues motivate our work.

(1) Deployment. PhishLang's "client-side" deployment is a local Python service that a browser extension queries. It requires installing TensorFlow and PyTorch (up to 2.5 GB), and the public installer is a Debian package. The authors considered in-browser inference with TensorFlow.js but expected it to be slower and less consistent across platforms. Since then, WebAssembly/WebGPU runtimes such as ONNX Runtime Web [10] and Transformers.js [11] have made in-browser transformer inference practical, and 8-bit quantization shrinks BERT-family models about 4× with little accuracy loss [12, 13]. To our knowledge, no one has measured whether a phishing-detecting MobileBERT can run inside a standard extension with no native install.

(2) Evaluation validity. Results in ML-for-security are known to be inflated by data snooping, sampling bias and temporally mixed splits [14, 15]. PhishLang trains and tests on a random split of Phishpedia, whose phishing samples carry 2019–2020 collection timestamps. Our inspection of the *released* training script found two further issues:

(a) Pages are truncated to their first 128 tokens during training, while inference takes the maximum over 128-token sliding windows. The model is therefore trained and deployed on different views of a page.

(b) Cross-validation indices computed on the shuffled training split are applied to the unshuffled full dataset. Replaying this logic at the paper's dataset sizes places 51–76% of the test pages inside each fold's training data.

If the published numbers were produced this way, they overstate generalization. It is therefore an open question whether PhishLang holds up on *current*, previously unseen phishing. PhreshPhish [16], a recent benchmark with a temporally later test split, leakage filtering and realistic base rates, makes this test feasible.

3 Problem Statement

Can a MobileBERT phishing detector run entirely inside the user's browser (no cloud, no blocklist, no native install) while keeping the accuracy it shows under a leakage-free, temporally realistic evaluation?

RQ1 Validity. How do PhishLang's precision, recall and F1 change when the leakage is removed and the model is tested on phishing pages collected years after its training data?

RQ2 Context. How much does the 128-token training window limit detection, and does training on the sliding-window view used at inference close the gap?

RQ3 Deployment. What accuracy, latency, memory and size trade-offs result from running an INT8-quantized MobileBERT inside a Chrome Manifest V3 (MV3) extension? We compare it with FP32 PyTorch and with PhishLang's local-server design.

4 Objectives and Goals

By the end of the semester we will deliver:

O1 A reproducible re-evaluation of the released PhishLang model and a retrained baseline under (i) a corrected random split and (ii) a temporal split (2019–20 $\rightarrow$ 2024–25). We will report precision, recall, F1 and PR-AUC, including at low, realistic phishing base rates.

O2 A context-window ablation: first 128 tokens vs. sliding-window training and inference vs. longer windows (up to 512 tokens).

O3 *PhishLang-Web*, an open-source MV3 extension that parses the live DOM and runs an INT8 ONNX MobileBERT locally via ONNX Runtime Web (WASM, plus WebGPU where available), with no network request for inference.

O4 A measurement study of O3 on a desktop and a low-end laptop: agreement with PyTorch, per-page latency, peak memory and package size. Our targets are $\geq$98% agreement with FP32, a median latency under 1 s on a laptop CPU, and a package under 50 MB.

5 Approach and Methodology

Phase A: Audit (RQ1–2). PhishLang's released parsed samples form the 2019–20 corpus. The PhreshPhish test split, parsed with the same upstream parser, forms the 2024–25 corpus. We remove exact and near-duplicate pages before splitting. We evaluate the released checkpoint as-is, then retrain it with the upstream hyper-parameters on a corrected split (three seeds, mean $\pm$ std). For RQ2 we retrain with each window strategy and compare them on both corpora.

Phase B: In-browser deployment (RQ3). We export the model to ONNX, apply dynamic INT8 quantization and verify parity in Python. We then port the parser to JavaScript over the live DOM and check that it matches the Python parser on saved pages. Tokenization and inference run in an extension worker or offscreen document with ONNX Runtime Web. We time each stage (parse, tokenize, infer) and record memory and package size. The baselines are PyTorch FP32 on the CPU (approximating PhishLang's local server) and ONNX FP32.

Ethics and safety. All phishing samples are stored HTML, handled offline in an isolated environment. We never visit live phishing URLs and collect no user data.

6 Expected Challenges

Challenge	Mitigation
Antivirus software silently and non-randomly quarantines phishing samples (we observed this), which biases the data.	Keep samples in an isolated Linux environment; check file counts and hashes against manifests before every run.
The data comes in different formats (raw vs. pre-parsed HTML), and near-duplicate phishing kits inflate scores.	Use one parser for all data, spot-check its output by hand, and deduplicate on the parsed text before splitting.
INT8 quantization may degrade MobileBERT's accuracy.	Quantize the linear layers only and compare with FP32 layer by layer; fall back to FP16 on WebGPU or keep sensitive layers in FP32.
MV3 limits: short-lived service workers, a CSP that needs `wasm-unsafe-eval`, and no remotely hosted code.	Bundle the runtime and model inside the extension; run inference in an offscreen document.
The live DOM differs from the static HTML after scripts run.	Measure parser agreement; report the differences as a finding, since the live view may expose DOM-manipulation evasions.
Seven weeks and two people.	Run Phases A and B in parallel. The minimum viable result is the temporal evaluation plus an FP32 in-browser prototype.

7 Expected Results and Significance

We hypothesize that the temporal evaluation will show a measurable drop from the reported 0.96 F1. This would quantify how quickly client-side language-model detectors age and how often they need retraining. We also expect an INT8 model of about a quarter of the FP32 size to run in the browser at interactive latency. That would remove PhishLang's multi-gigabyte native dependency and make the detector installable from an extension store on any Chromium browser. The broader impact is a zero-install, offline detector that keeps page content and browsing history on the device and removes blocklist latency, narrowing the gap between academic detectors and deployable browser security. We will release our corrected splits, scripts and extension.

8 Resources

Hardware: a workstation (AMD Ryzen 7 9700X, 32 GB RAM, NVIDIA RTX 5070 12 GB) and a low-end laptop for client-side benchmarks. **Data:** PhishLang's released parsed samples and checkpoint; PhreshPhish (CC BY 4.0). **Software:** PyTorch, Hugging Face Transformers, ONNX Runtime (Python and Web), Transformers.js and Chrome (MV3). All are free, and no cloud budget is required.

9 Timeline

Week (2026)	Milestone
Oct 26 – Nov 1	Proposal submitted; isolated data environment; upstream code and model reproduced.
Nov 2 – Nov 8	Leakage-free split; retrained baseline; PhreshPhish parsed and deduplicated.
Nov 9 – Nov 15	Temporal evaluation (RQ1); context-window ablation (RQ2).
Nov 16 – Nov 22	ONNX export, INT8 quantization, Python parity; JavaScript parser.
Nov 23 – Nov 29	MV3 extension prototype; parser and model parity in the browser.
Nov 30 – Dec 6	Benchmarks on two machines (RQ3); 30-minute presentation.
Dec 7 – Dec 13	Final report (IEEE, $\leq$8 pages); code and artifact release.

References

[1] A. Oest, Y. Safaei, P. Zhang, B. Wardman, K. Tyers, Y. Shoshitaishvili, and A. Doupé, "PhishTime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 379–396.

[2] S. Saha Roy and S. Nilizadeh, "PhishLang: A real-time, fully client-side phishing detection framework using MobileBERT," in *Network and Distributed System Security Symposium (NDSS)*, 2026, arXiv:2408.05667. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/phishlang-a-real-time-fully-client-side-phishing-detection-framework-using-mobilebert/

[3] R. Pourmohamad, S. Wirsz, A. Oest, T. Bao, Y. Shoshitaishvili, R. Wang, A. Doupé, and R. A. Bazzi, "Deep dive into client-side anti-phishing: A longitudinal study bridging academia and industry," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2024, pp. 638–653.

[4] T. Koide, H. Nakano, and D. Chiba, "ChatPhishDetector: Detecting phishing sites using large language models," *IEEE Access*, vol. 12, pp. 154 381–154 400, 2024.

[5] Y. Li, C. Huang, S. Deng, M. L. Lock, T. Cao, N. Oo, H. W. Lim, and B. Hooi, "KnowPhish: Large language models meet multimodal knowledge graphs for enhancing reference-based phishing detection," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 793–810.

[6] T. Cao, C. Huang, Y. Li, W. Huilin, A. He, N. Oo, and B. Hooi, "PhishAgent: A robust multimodal agent for phishing webpage detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 27, 2025, pp. 27 869–27 877.

[7] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: a compact task-agnostic BERT for resource-limited devices," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2158–2170.

[8] Y. Lin, R. Liu, D. M. Divakaran, J. Y. Ng, Q. Z. Chan, Y. Lu, Y. Si, F. Zhang, and J. S. Dong, "Phishpedia: A hybrid deep learning based approach to visually identify phishing webpages," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 3793–3810.

[9] B. Montaruli, L. Demetrio, M. Pintor, L. Compagna, D. Balzarotti, and B. Biggio, "Raze to the ground: Query-efficient adversarial HTML attacks on machine-learning phishing webpage detectors," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 233–244.

[10] ONNX Runtime developers, "ONNX Runtime (including ONNX Runtime Web)," https://onnxruntime.ai/, accessed: 2026-10-26.

[11] Hugging Face, "Transformers.js: State-of-the-art machine learning for the web," https://github.com/huggingface/transformers.js, accessed: 2026-10-26.

[12] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, "Q8BERT: Quantized 8bit BERT," in *Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing – NeurIPS Edition (EMC2-NIPS)*, 2019, pp. 36–39.

[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[14] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988.

[15] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 729–746.

[16] T. Dalton, H. Gowda, G. Rao, S. Pargi, A. H. Khodabakhshi, J. Rombs, S. Jou, and M. Marwah, "PhreshPhish: A real-world, high-quality, large-scale phishing website dataset and benchmark," arXiv:2507.10854, 2025.